Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Регламент обновления

Подготовка, обновление, проверка и откат · версия 1.0.0

30 октября 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Когда применять 4

3 Перед началом 5

4 Подготовка 6

4.1 1. Прочитайте, что меняется 6

4.2 2. Запишите текущее состояние 6

4.3 3. Проверьте место и конфигурацию 6

4.4 4. Подготовьте версию для закрытого контура 7

4.5 5. Согласуйте окно 7

5 Обновление 8

5.1 1. Начало работ 8

5.2 2. Получите новую версию 8

5.3 3. Соберите образы 8

5.4 4. Остановите приём запросов 9

5.5 5. Сделайте внеочередную копию 9

5.6 6. Запустите новую версию 9

5.7 7. Дождитесь старта 9

6 Проверка 11

6.1 1. Контейнеры, версия и пробы 11

6.2 2. Миграции 11

6.3 3. Метрики 11

6.4 4. Вход, график, журналы 11

6.5 5. Журнал сервера 12

6.6 6. На следующий день 12

7 Откат 13

7.1 1. Сохраните сведения для разработчиков 13

7.2 2. Верните прежнюю версию файлов 13

7.3 3. Решите, нужно ли восстанавливать базу 13

7.4 4. Запустите прежнюю версию 14

7.5 5. Сообщите о результате 14

8 Особые шаги 1.0.0 15

9 Запись в журнал эксплуатации 19

1 О документе

Регламент обновления «Планировщика отпусков» до новой версии (E52-04): подготовка, работы в окне, проверка после обновления, откат и особые шаги перехода на версию 1.0.0. Справочник по командам и переменным — «Руководство по развёртыванию» (`docs/manuals/deployment-guide.md`), разделы «Установка», «Настройка», «Обновление», «Резервное копирование» и «Проверка»; этот регламент описывает порядок действий и критерии «готово».

Все команды выполняются на узле стенда из каталога, где лежат `docker-compose.yml` и `.env`. Время в журналах контейнеров и в базе данных — UTC.

2 Когда применять

СИТУАЦИЯ	ПРИМЕНЯТЬ
Вышла новая версия (тег `vX.Y.Z` или дистрибутив `vacation-planner-X.Y.Z`), в том числе исправляющая	Да, весь регламент
Переход с 0.6.0 на 1.0.0	Да, плюс раздел «Особые шаги 1.0.0»
Изменилась только настройка в `.env` (почта, единый вход, `WEB_CONCURRENCY`)	Нет: `$C --profile full up -d server` — сервис пересоздаётся с новыми переменными
Основной сервер PostgreSQL недоступен	Нет — `failover.md`

3 Перед началом

ЧТО НУЖНО	ПОДРОБНОСТИ
Доступ	SSH к узлу стенда, право запускать `docker`, чтение и правка `.env`
Кто участвует	Администратор стенда — выполняет; сотрудник кадровой службы или администратор приложения — проверяет интерфейс; ответственный за систему — решает об откате
Время	Окно работ 1 час вне рабочего времени: простой обычно 10–20 минут, остальное — запас на откат. Не в последний день кампании планирования
Получение версии	Доступ к репозиторию, npm и Docker Hub для сборки — или дистрибутив версии с образами для закрытого контура
Утилиты на узле	`git` (стенд из исходных текстов), `curl`, `jq`

Задайте переменную `C` один раз в сеансе — от неё зависят все команды регламента:

```
C="docker compose"                                                  # стенд без
реплики
C="docker compose -f docker-compose.yml -f docker-compose.replica.yml"     # стенд с
репликой и архивом WAL
```

На стенде с репликой нельзя запускать `docker compose up` без второго файла: `postgres` будет пересоздан без архивирования WAL.

4 Подготовка

Выполняется за 1–3 рабочих дня до окна. Стенд при этом работает.

4.1 1. Прочитайте, что меняется

1. `docs/versions.md` новой версии — разделы «Что увидит пользователь» и «Для эксплуатации»; `CHANGELOG.md` — раздел версии: особые шаги, изменения схемы, новые переменные.
2. Для перехода на 1.0.0 — раздел «Особые шаги 1.0.0» этого регламента.
3. Новые миграции схемы (стенд из исходных текстов):

```
git fetch --tags
git diff --name-only <текущий тег> <новый тег> -- server/prisma/migrations
```

Ожидаемый результат: перечень новых каталогов миграций; пусто — схема не меняется. Для перехода с 0.6.0 на 1.0.0 новых миграций нет (последняя — `20261031120000_planning`).

4.2 2. Запишите текущее состояние

```
$C exec server node -p "require('./package.json').version"     # текущая версия
git describe --tags                                             # тег исходных текстов
(стенд из git)
$C --profile full images                                        # образы работающих
контейнеров
docker volume ls | grep -E 'db-data|db-backups|db-replica|wal-archive'
tar czf ~/vp-before-update-$(date +%Y%m%d).tar.gz .env docker-compose*.yml deploy
chmod 600 ~/vp-before-update-*.tar.gz
```

Ожидаемый результат: версия, тег и образы записаны в журнал эксплуатации; у томов общий префикс — имя проекта compose (по умолчанию имя каталога стенда). Архив с `.env` содержит секреты — храните его как секрет.

Все дальнейшие команды запускайте из этого же каталога: из другого каталога compose поднимет стенд с новыми пустыми томами `<другой проект>_db-data`.

4.3 3. Проверьте место и конфигурацию

```
df -h . "$(docker info -f '{{.DockerRootDir}}')"
docker system df
$C exec backup ls -lt /backups | head -3
git status --short
```

Ожидаемый результат: свободно не меньше двойного размера последней копии плюс место под новые образы (размер нынешних — в `docker system df`); `git status` пуст или показывает только ваш `docker-compose.override.yml` — локальные правки файлов репозитория перенесите в него или в `.env`: `git checkout` новой версии откажет или унесёт их с собой.

4.4 4. Подготовьте версию для закрытого контура

Если у узла нет доступа к npm и Docker Hub, загрузите образы из дистрибутива заранее — работающий стенд это не затрагивает:

```
cd <каталог дистрибутива>/vacation-planner-<версия>
sha256sum -c SHA256SUMS                                  # все файлы: OK
docker load < vacation-planner-<версия>-images.tar.gz    # server, web и postgres:17-alpine
docker image ls 'vacation-planner-*'
```

Ожидаемый результат: `vacation-planner-server:<версия>` и `vacation-planner-web:<версия>` в списке образов.

4.5 5. Согласуйте окно

1. Сообщите кадровой службе время простоя и изменения для пользователей из `docs/versions.md` (для 1.0.0 — таблица «Особые шаги 1.0.0»).
2. Предупредите дежурных по мониторингу: в окне сработают `VacationApiDown`, `VacationDatabaseDown`.
3. Для 1.0.0 выполните подготовку из колонки «До обновления» раздела «Особые шаги 1.0.0» — прежде всего проверку `JWT_SECRET`: без него после смены версии не выполнится ни одна команда `docker compose`.

5 Обновление

5.1 1. Начало работ

```
$C --profile full ps
```

Ожидаемый результат: все сервисы `running`, `postgres` — `healthy`. Запишите время начала. Если стенд уже неисправен — сначала `alerts.md` или `diagnostics.md`, обновление переносится.

5.2 2. Получите новую версию

Работающий стенд на этом шаге не меняется: контейнеры продолжают работать на прежних образах.

Стенд из исходных текстов:

```
git fetch --tags && git checkout v1.0.0
```

Стенд из дистрибутива: замените `docker-compose.yml`, `docker-compose.replica.yml` и каталог `deploy/` файлами из `install/` нового дистрибутива; `.env` оставьте свой.

В обоих случаях задайте в `.env` версию образов — `VP_VERSION=1.0.0`. С ней образы получают имена `vacation-planner-server:1.0.0` и `vacation-planner-web:1.0.0`, а образы прежней версии остаются под своим тегом для отката; без неё каждая сборка перезаписывает тег `local`.

```
grep '^VP_VERSION=' .env
$C --profile full config --quiet
```

Ожидаемый результат: `VP_VERSION=1.0.0`; `config` молчит. Сообщение «required variable JWT_SECRET is missing a value» — секрет не задан (раздел «Особые шаги 1.0.0»).

5.3 3. Соберите образы

Только для стенда из исходных текстов с доступом к npm и Docker Hub; образы из дистрибутива уже загружены.

```
$C --profile full build
```

Ожидаемый результат: сборка `server` и `web` завершилась без ошибок; `docker image ls 'vacation-planner-*'` показывает образы с тегом `VP_VERSION`. Стенд всё ещё работает на прежних образах.

5.4 4. Остановите приём запросов

```
$C --profile full stop web server
```

Ожидаемый результат: `web` и `server` — `exited`. С этого момента идёт простой; копия на следующем шаге совпадёт с состоянием базы на момент обновления, и откат с восстановлением не потеряет изменений пользователей.

5.5 5. Сделайте внеочередную копию

```
$C exec backup sh -c 'pg_dump -Fc -f /backups/vacation-manual-$(date +%Y%m%d-%H%M).dump'
$C exec backup ls -lh /backups | tail -3
$C exec backup sh -c 'pg_restore -l /backups/<файл>.dump | head -5'
$C cp backup:/backups/<файл>.dump ./
```

Ожидаемый результат: новый файл `vacation-manual-…dump` размером не меньше последних ежедневных копий; `pg_restore -l` печатает заголовок оглавления («Archive created at …») — копия читается; файл скопирован на узел. Перенесите его на хранилище вне сервера средствами организации и запишите имя файла в журнал. Копии `vacation-manual-*` в томе удаляются вместе с ежедневными через `BACKUP_KEEP_DAYS` дней.

5.6 6. Запустите новую версию

```
$C --profile full up -d
```

Ожидаемый результат: compose пересоздаёт `server` и `web` (и сервисы, у которых изменилась конфигурация, — при переходе на 1.0.0 это `postgres`, а на стенде с репликой и `postgres-replica`). Стенд из исходных текстов, где шаг 3 пропущен, запускается одной командой руководства — `$C --profile full up -d --build`; сборка тогда идёт во время простоя.

Если на стенде работает Prometheus из профиля `monitoring`, перечитайте правила оповещений новой версии — файлы `deploy/monitoring/*.yml` подключены в контейнер и читаются при старте:

```
$C --profile monitoring restart prometheus
```

5.7 7. Дождитесь старта

```
$C logs -f --no-log-prefix server
```

Ожидаемый результат: сообщение Prisma «No pending migrations to apply.» или «All migrations have been successfully applied.», затем строка `"msg":"Запуск N процессов API"` (если процессов больше одного) и `"msg":"API планировщика отпусков слушает порт 8080; документация:`

`/api/docs"` от каждого процесса. Выход — Ctrl+C.

Ошибка миграции, `"msg":"API не запустился"` или `"msg":"Процессы API падают слишком часто — останов"` — контейнер `server` останавливается (политики перезапуска в compose нет). Сохраните журнал и переходите к разделу «Откат».

6 Проверка

6.1 1. Контейнеры, версия и пробы

```
$C --profile full ps                                                    # все running,
postgres healthy
$C exec server node -p "require('./package.json').version"              # 1.0.0
$C exec server wget -qO- http://localhost:8080/healthz                  # {"ok":true}
$C exec server wget -qO- http://localhost:8080/readyz                   #
{"ok":true,"db":"up"}
curl -s http://localhost/api/v1/employees -o /dev/null -w '%{http_code}\n' # 401 — API за
nginx отвечает
```

6.2 2. Миграции

```
$C exec postgres psql -U vacation -d vacation -c "select migration_name, finished_at from
_prisma_migrations order by started_at desc limit 3"
```

Ожидаемый результат: последняя миграция — последняя из каталога миграций версии, у всех `finished_at` заполнено.

6.3 3. Метрики

```
$C exec -T server sh -c 'wget -qO- --header "Authorization: Bearer $METRICS_TOKEN"
http://127.0.0.1:8080/metrics' \
  | grep -E '^vp_(db_up|process_count)'
```

Ожидаемый результат: `vp_db_up 1`, `vp_process_count` равно числу процессов (`WEB_CONCURRENCY` или числу ядер). В Prometheus («Status → Targets») задание `vacation-planner` — `UP`; во вкладке «Alerts» нет сработавших правил, кроме ожидаемых в окне.

6.4 4. Вход, график, журналы

Сотрудник кадровой службы или администратор приложения:

1. Входит в интерфейс (по паролю и, если настроен, через единый вход).
2. Открывает «График отпусков» — график строится; «Справочники → Производственный календарь» — календарь года на месте.
3. Открывает «Журнал аудита → Входы» — его вход только что записан.
4. Для 1.0.0 — проверки из таблицы «Особые шаги 1.0.0».

6.5 5. Журнал сервера

```
$C logs --no-log-prefix --since 10m server | jq -cR 'fromjson? | select(.level=="error")' |
head
```

Ожидаемый результат: пусто. Строки с `"msg":"unhandled error"` — стек и `reqId` передаются разработчику (`diagnostics.md`), решение об откате — за ответственным.

6.6 6. На следующий день

```
$C exec backup ls -lt /backups | head -3                    # ночная копия новой версии
$C exec postgres psql -U vacation -d vacation -c "select name, period, ran_at from job_runs
order by ran_at desc limit 5"
```

Ожидаемый результат: свежая ежедневная копия; ежедневные задачи (`login-retention`, `data-retention` и др.) отметились за сегодня.

7 Откат

Выполняется, если `server` не стартует после обновления, проверка нашла дефект, который мешает работе, или данные искажены. Решение принимает ответственный за систему; откат укладывается в то же окно.

7.1 1. Сохраните сведения для разработчиков

```
$C logs --no-log-prefix --timestamps server > server-failed-$(date +%Y%m%d-%H%M).log
$C --profile full stop server web
```

7.2 2. Верните прежнюю версию файлов

Стенд из исходных текстов — `git checkout <прежний тег>`. Стенд из дистрибутива — верните файлы из архива подготовки:

```
tar xzf ~/vp-before-update-<дата>.tar.gz docker-compose.yml docker-compose.replica.yml
deploy
```

В `.env` верните прежнее значение `VP_VERSION` (или удалите строку, если её не было). `JWT_SECRET` оставьте заданным: файл compose 0.6.0 без него подставляет известное значение по умолчанию.

7.3 3. Решите, нужно ли восстанавливать базу

СЛУЧАЙ	ДЕЙСТВИЕ
Новая версия применила миграции	Восстановите копию шага «Обновление → 5» — раздел «Полное восстановление» в `backup-restore.md` : миграции не откатываются
Миграций не было (1.0.0 → 0.6.0), данные целы	Не восстанавливайте: прежняя версия работает с текущей базой, изменения пользователей после обновления сохранятся
Данные искажены новой версией или нужно вернуть строки, удалённые задачей `data-retention`	Восстановите копию шага «Обновление → 5»; изменения после неё будут потеряны

7.4 4. Запустите прежнюю версию

```
$C --profile full up -d            # образы прежней версии сохранились (тег VP_VERSION; у
0.6.0 — образы проекта без тега версии)
$C --profile full up -d --build    # иначе — сборка прежней версии из исходных текстов
```

Ожидаемый результат: проверка из раздела «Проверка» проходит с прежним номером версии; `$C --profile full images` показывает те же образы, что записаны на шаге «Подготовка → 2».

7.5 5. Сообщите о результате

Кадровой службе — что стенд работает на прежней версии; разработчикам — журнал из шага 1, версию, шаг, на котором произошёл сбой, и вывод команд.

8 Особые шаги 1.0.0

ИЗМЕНЕНИЕ	ДО ОБНОВЛЕНИЯ	ПОСЛЕ ОБНОВЛЕНИЯ
`JWT_SECRET` обязателен: в `docker-compose.yml` больше нет значения по умолчанию, compose отказывается выполнять любую команду (`up` , `exec` , `logs`) с сообщением «required variable JWT_SECRET is missing a value»	`awk '/^JWT_SECRET=/ { print length($0) - 11 }' .env` — длина не меньше 32; иначе задайте `JWT_SECRET=<результат openssl rand -base64 48>` . Если стенд до 1.0.0 работал без `JWT_SECRET` в `.env` , на нём действовало значение из репозитория и access-токен любого сотрудника можно было подделать (самооценка ИБ, F-1) — передайте период работы в службу ИБ для разбора по `incident.md`	Новый секрет делает недействительными только access-токены: открытые вкладки один раз получают `401` , сами обновляют токен и продолжают работу; входить заново не нужно

ИЗМЕНЕНИЕ	ДО ОБНОВЛЕНИЯ	ПОСЛЕ ОБНОВЛЕНИЯ
PostgreSQL опубликован только на `127.0.0.1:5432`	Выясните, кто подключается к базе извне узла: `$C exec postgres psql -U vacation -d vacation -c "select client_addr, application_name, count(*) from pg_stat_activity where datname = 'vacation' group by 1, 2"` — всё, кроме соединений `server` и `backup`, сверьте с владельцами (системы копирования, отчётности, инструменты администраторов). После обновления им нужен доступ с узла стенда или SSH-туннель. До 1.0.0 порт был открыт на всех интерфейсах с паролем из репозитория (F-2) — если узел был доступен из сети, служба ИБ проверяет журналы межсетевого экрана	`$C port postgres 5432` — `127.0.0.1:5432`
Образы `server` и сборки `web` — на `node:22-alpine`	Для сборки нужен доступ к Docker Hub (`node:22-alpine`, `nginx:1.27-alpine`) и npm; в закрытом контуре — образы из дистрибутива («Подготовка → 4»)	`docker image ls 'vacation-planner-*'` — образы версии 1.0.0
Сервисы стенда — с `restart: unless-stopped`: после перезагрузки узла стенд поднимается сам	Ничего: политика применяется при пересоздании контейнеров на шаге запуска. Контейнер, остановленный командой `stop`, остаётся остановленным и после перезагрузки	`docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' $($C ps -q server)` — `unless-stopped`

ИЗМЕНЕНИЕ	ДО ОБНОВЛЕНИЯ	ПОСЛЕ ОБНОВЛЕНИЯ
`DB_POOL_SIZE`, `TRUST_PROXY`, `EVENTS_POLL_MS` из `.env` теперь доходят до `server` (до 1.0.0 compose их не передавал — действовали значения по умолчанию); пароль БД — переменная `POSTGRES_PASSWORD` (по умолчанию прежний)	Проверьте эти строки в `.env`: если они задавались раньше «на всякий случай», после обновления они начнут действовать. `TRUST_PROXY` с публичным адресом прокси — именно так, как описано в руководстве	`$C exec server printenv TRUST_PROXY EVENTS_POLL_MS`
Ежедневная задача `data-retention`: удаляет истёкшие refresh-токены и токены завершённых сеансов (через сутки), отправленные и неотправленные письма старше 90 дней, прочитанные уведомления старше года, отметки задач старше 90 дней. Заявки, история, извещения, утверждённые графики и журнал изменений не удаляются. Первый запуск — в первые минуты после старта, а не ночью	Оцените объём: `$C exec postgres psql -U vacation -d vacation -c "select (select count(*) from email_outbox where status in ('sent','failed') and coalesce(sent_at, created_at) < now() - interval '90 days') as mail, (select count(*) from notifications where is_read and created_at < now() - interval '365 days') as notifications, (select count(*) from refresh_tokens where expires_at < now() - interval '1 day') as tokens"`. Если письма нужны для разбора жалоб — выгрузите их до обновления; копия шага «Обновление → 5» их тоже сохраняет	`$C logs --no-log-prefix server | grep 'Сроки хранения'` — строка `"Сроки хранения: удалено {…}"` (её нет, если удалять было нечего); `job_runs` содержит `data-retention` за сегодня. Большое удаление пачками по 5000 строк кратко нагружает WAL — на стенде с репликой возможен короткий `VacationReplicaLagging`

ИЗМЕНЕНИЕ	ДО ОБНОВЛЕНИЯ	ПОСЛЕ ОБНОВЛЕНИЯ
Карточка коллеги сокращена: сотрудник без служебной роли видит у коллег только справочные поля, без логина, почты, норм дней, табельного номера и служебных прав; поиск по табельному номеру — только кадровой службе. Полная карточка — себе, своему руководителю, кадровой службе и администратору	Предупредите кадровую службу и руководителей: обращения «пропала почта коллеги» ожидаемы	Сотрудник без служебной роли открывает карточку коллеги — логина и почты нет
Неверный старый пароль при смене пароля считается неудачной попыткой входа: после порога учётная запись блокируется, как при входе. При смене и выдаче пароля сотрудник получает уведомление `password_changed`	Предупредите кадровую службу: блокировки после неудачной смены пароля ожидаемы, снимаются как обычно («Учётная запись → Снять блокировку»)	В «Журнале аудита → Входы» такая попытка — «неверный пароль» без адреса и браузера
Журнал `server` — строки JSON (`LOG_FORMAT=json` в compose) с кодом запроса; PostgreSQL пишет операторы дольше `PG_SLOW_MS` (1000 мс) и загружает `pg_stat_statements`	Если сборщик журналов организации разбирал текстовый журнал 0.6.0 — перенастройте разбор на JSON	`$C logs --no-log-prefix --tail 3 server` — строки JSON; `$C exec postgres psql -U vacation -d vacation -c "show shared_preload_libraries"` — `pg_stat_statements`
`server` останавливается штатно по SIGTERM: запросы и потоки событий завершаются, останов занимает до 15 секунд	—	—

9 Запись в журнал эксплуатации

ПОЛЕ	ПРИМЕР
Дата и окно работ	2026-11-20, 19:00–20:00 МСК
Исполнитель и проверяющий	Администратор стенда; сотрудник кадровой службы
Версия было → стало	0.6.0 → 1.0.0; образы `vacation-planner-server:1.0.0`, `vacation-planner-web:1.0.0`
Копия перед обновлением	`vacation-manual-20261120-1605.dump` (UTC), размер, место хранения вне сервера
Простой	С остановки `web` до успешной проверки входа, минут
Результат проверки	Пробы, миграции, метрики, вход, график, журнал — пройдено / замечания
Отклонения и откат	Нет / что произошло, принятое решение, ссылка на журнал `server-failed-…log`